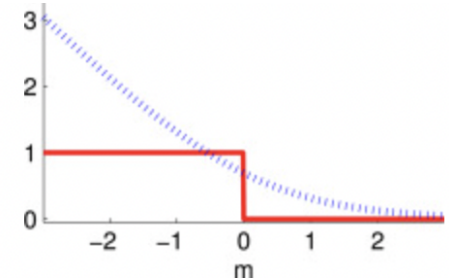
Regularisation

Procheta Sen

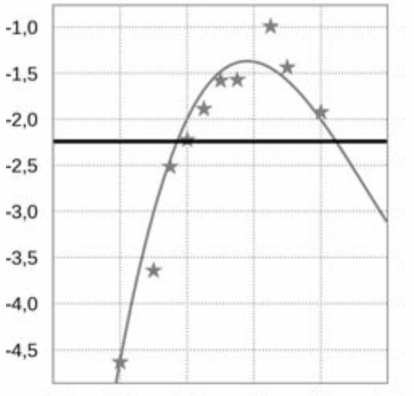


Introduction to Regularisation

- Regularisation is a process of reducing overfitting in a model by constraining it (reducing the complexity/no. of parameters).
- For classifiers that use a weight vector, regularisation can be done by minimising the norm (length) of the weight vector.
- Several popular regularisation methods exist.
  - L2 regularisation (ridge regression or Tikhonov regularisation).
  - L1 regularisation (Lasso regression).
  - L1+L2 regularisation (mixed regularisation).

Approximation of a Polynomial *d*

- Approximation of $f(x) = x^3 - 4x^2 + 3x - 2$ by a polynomial of degree d.
- The dataset $\mathcal{D} = \{x_i, y_i\}_{i=1}^n$ is sampled from the polynomial $f(x) = x^3 - 4x^2 + 3x - 2$ with some noise.



Approximation Loss Function

- We want to approximate the unknown function $f(x)$ by a polynomial of degree d.
- $\hat{y}(x, \overline{W}) = W_0 + \sum_{j=1}^{d} w_j x^j = (1, x^1, x^2 \dots x^d).\overline{W}$
- such that the following loss function (residual sum of squares (RSS)) is minimised $L(\mathcal{D}, \overline{W}) = \sum_{i=1}^{n} (\hat{y}(x_i, \overline{W}) - y_i)^2$

Approximation Example

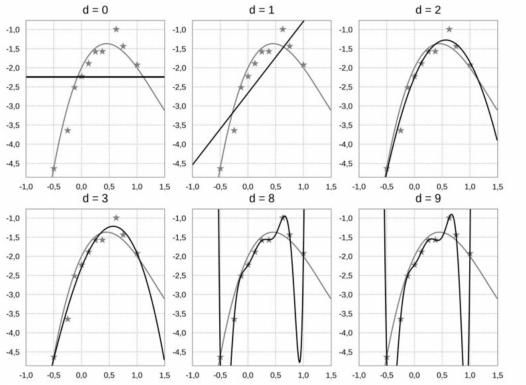


Figure: Approximation with different values of d

Approximation Formulations

- $f_0(x) = -2,2393$
- $f_1(x) = -2,6617 + 1,8775x$
- $f_2(x) = -2,2528 + +3,4604x - 3,0603x^2$
- $f_8(x) = -2,2324 + 2,2326x + 6,2543x^2 + 15,599gx^3 - 239,9751x^4 + 322,8516x^4 + 621,0952x^6 - 1478,6505x^7 + 750,9032x^8$
- $f_9(x) = -2,22 + 2,01x + 4,88x^2 + 31,13x^3 - 230,31x^4 + 103,72x^5 + 869,22x^6 - 966,67x^7 - 319,31x^8, 505,64x^9$
- The parameters grow!
- Let's try to restrict their growth.
- $L(\mathcal{D}, \overline{W}) = \sum_{i=1}^{n}(\hat{y}(x_i, \overline{W}) - y_i)^2 + \lambda||W||^2$

Approximation with Different values of λ

$$
\begin{aligned}
f_{\lambda=0}(x) = &-2{,}22 + 2{,}01x + 4{,}88x^2 + 31{,}13x^3 - 230{,}31x^4 + \\
&+ 103{,}72x^5 + 869{,}22x^6 - 966{,}67x^7 - 319{,}31x^8 + 505{,}64x^9, \\
f_{\lambda=0{,}01}(x) = &-2{,}32 + 3{,}40x - 2{,}33x^2 + 0{,}05x^3 - 0{,}51x^4 - \\
&- 0{,}29x^5 - 0{,}22x^6 - 0{,}06x^7 + 0{,}09x^8 + 0{,}24x^9, \\
f_{\lambda=1}(x) = &-2{,}46 + 1{,}45x - 0{,}19x^2 + 0{,}22x^3 - 0{,}13x^4 - \\
&- 0{,}05x^5 - 0{,}14x^6 - 0{,}13x^7 - 0{,}16x^8 - 0{,}16x^9.
\end{aligned}
$$

Figure: Equation with different values of λ

f(x) Plot With Different Values of Lambda

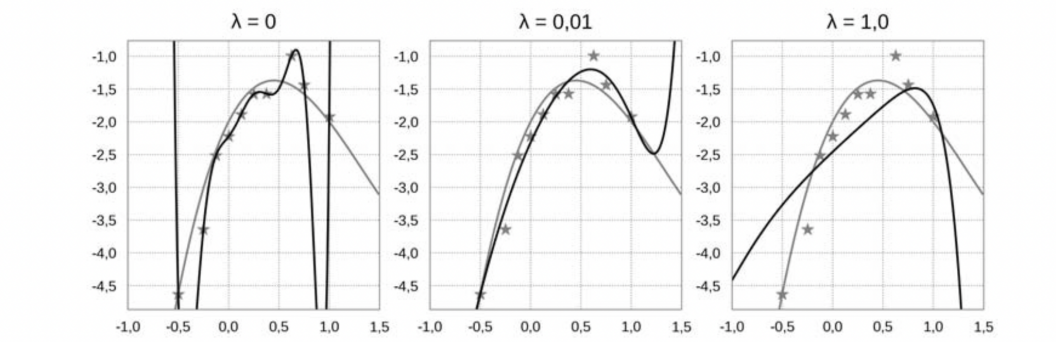


Figure: Caption

L2 Regularisation

- Let us denote by $L(\mathcal{D}, \mathbf{W})$ the loss of classifying the dataset $\mathcal{D}$ using the model represented by the weight vector $\mathbf{W}$.
- We would like to impose L2 regularisation on $\mathbf{W}$.
- The overall objective to minimise can then be written as follows:

$$J(\mathcal{D}, \mathbf{W}) = L(\mathcal{D}, \mathbf{W}) + \lambda \|\mathbf{W}\|^2 = L(\mathcal{D}, \mathbf{W}) + \lambda \sum_{i=1}^{d} w_i^2$$

- Here λ is called the **regularisation coefficient** and is usually set via **cross-validation**.
- The gradient of the overall objective simply becomes the sum of the loss-gradient and the scaled weight vector $\mathbf{W}$.

Examples

- Note that SGD update rule for minimising a loss multiplies the loss gradient by a negative learning rate (μ).
- Therefore, the L2 regularised update rules will have a $-2\mu\lambda\mathbf{W}$ term as shown in the following examples.

L2 Regularised Perceptron Update Rule

$$\mathbf{W} \leftarrow \mathbf{W} - \mu\left(-y_i\mathbf{X}_i + 2\lambda\mathbf{W}\right)$$

$$= \mathbf{W} + \mu\, y_i\mathbf{X}_i - 2\mu\lambda\mathbf{W}$$

$$= \mathbf{W} + y_i\mathbf{X}_i - 2\lambda\mathbf{W} \quad (\text{for } \mu = 1)$$

$$= (1 - 2\lambda)\mathbf{W} + y_i\mathbf{X}_i$$

How to set λ

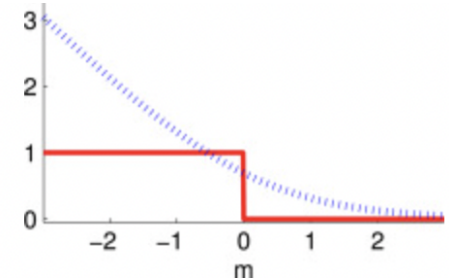
- Split your training dataset into **training** and **validation** parts (e.g., 80%–20%).
- Try different values for λ (typically in the logarithmic scale), e.g.

$$\lambda = 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 10^{2}, 10^{3}, 10^{4}, 10^{5}$$

- Train a different classification model for each λ and select the value that gives the best performance (e.g., accuracy) on the validation data.

Introduction to KNN

Procheta Sen



World's simplest classifier!

- Given a training dataset

$$D_{\text{train}}$$

of N instances $(\mathbf{X}, y)$, simply *remember* the entire N instances in memory (or hard-disk): **memory-based learning**.

- When we want to classify a test (previously unseen, not in D_{train}) instance $\mathbf{X}'$, we would simply check whether $\mathbf{X}'$ is in D_{train}.
- If $\mathbf{X}' \in D_{\text{train}}$, then return the label of $\mathbf{X}'$.
- Otherwise, make a random guess.

The Nearest Neighbour classifier

- **Training:** store entire training set D_{train}.
- **Classification:** for an input object $\mathbf{X}'$, find in the training set the *closest* object $\mathbf{X}$ to $\mathbf{X}'$ and classify $\mathbf{X}'$ the same as $\mathbf{X}$.

The k-Nearest Neighbour classifier

- **Training:** store entire training set.
- **Classification:** for an input object $\mathbf{X}'$,
  - find in the training set the k closest (nearest) objects to $\mathbf{X}'$,
  - find the majority label among those nearest neighbours,
  - the majority label is predicted for the test instance $\mathbf{X}'$.

Example

5-NN: find 5 closest to the black point. 3 blue and 2 red, so predict blue

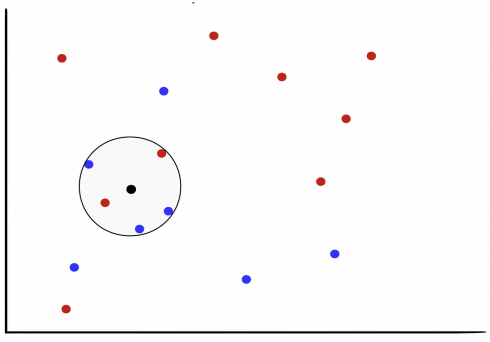
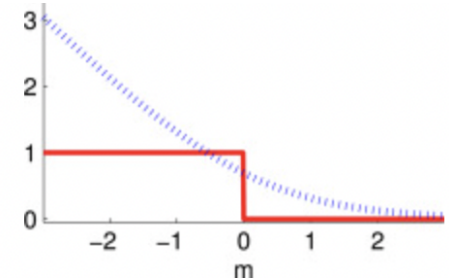


Figure: Caption

Measures of Similarity/Distance

Procheta Sen



Measures of similarity/distance

General problem

Given two objects $\bar{X}$ and $\bar{Y}$ determine the value of the **similarity** or **distance** between the two objects.

Similarity functions: larger values imply greater similarity

Distance functions: smaller values imply greater similarity

In some domains, such as spatial data, it is more natural to talk about distance functions, whereas in other domains, such as text, it is more natural to talk about similarity functions.

Similarity and distance functions are often expressed in closed form (easy to compute formulas), but in some domains, such as time-series data, they are defined algorithmically and may be computationally expensive.

Measures of similarity/distance for numerical data

Euclidean distance between the vectors $\bar{X}$ and $\bar{Y}$.

$$\mathrm{EucDist}(\bar{X}, \bar{Y}) = \|\bar{X} - \bar{Y}\|_2 = \sqrt{\sum_{i=1}^{d}(x_i - y_i)^2} = \sqrt{(\bar{X} - \bar{Y})^T(\bar{X} - \bar{Y})}$$

where $\|\bar{T}\|_2 = \sqrt{\sum_{i=1}^{d} t_i^2}$ denotes the L^2-norm of the vector $\bar{T} = (t_1, t_2, \ldots, t_d)^T$.

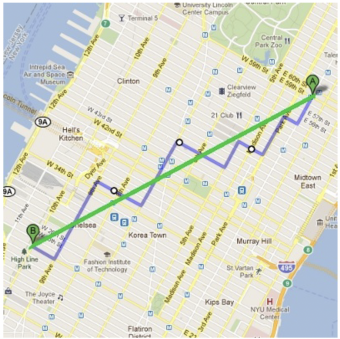
Measures of similarity/distance for numerical data

Manhattan Distance between the vectors $\bar{X}$ and $\bar{Y}$.

$$\mathrm{ManDist}(\bar{X}, \bar{Y}) = \|\bar{X} - \bar{Y}\|_1 = \sum_{i=1}^{d} |x_i - y_i|$$

where $\|\bar{T}\|_1 = \sum_{i=1}^{d} |t_i|$ denotes the L^1-norm of the vector $\bar{T} = (t_1, t_2, \ldots, t_d)^T$.

Example of Manhattan Distance



$EucDist(A, B)$

$ManDist(A, B)$

Figure: Demonstration of Manhattan and Euclidian Distance

Vector norms

- "norm" is a mathematical concept that expresses the "size/length" of a vector
- Popular vector norms in data mining are as follows

$$L^1\text{-norm} \quad \|\bar{X}\|_1 = \sum_{i=1}^{d} |x_i|$$

$$L^2\text{-norm} \quad \|\bar{X}\|_2 = \sqrt{\sum_{i=1}^{d} x_i^2}$$

$$L^0\text{-norm} \quad \|\bar{X}\|_0 = \text{number of non-zero elements in } \bar{X}$$

$$L^\infty\text{-norm} \quad \|\bar{X}\|_\infty = \max\{|x_1|, |x_2|, \ldots, |x_d|\}$$

From vector norms to distances

From a vector norm, one can construct a distance function between pairs of vectors (say, $\bar{X}$ and $\bar{Y}$) as the norm of their difference.

Vector norms

L^1-norm $\|\bar{X}\|_1 = \sum_{i=1}^{d} |x_i|$

L^2-norm $\|\bar{X}\|_2 = \sqrt{\sum_{i=1}^{d} x_i^2}$

L^0-norm $\|\bar{X}\|_0 =$ no. of non-zero elements in $\bar{X}$

L^∞-norm $\|\bar{X}\|_\infty = \max\{|x_1|, \ldots, |x_d|\}$

Distances from norms

L^1-distance $\|\bar{X} - \bar{Y}\|_1 = \sum_{i=1}^{d} |x_i - y_i|$

L^2-distance $\|\bar{X} - \bar{Y}\|_2 = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}$

Measures of similarity/distance for numerical data

Cosine similarity

Let $\bar{X} = (x_1, x_2, \ldots, x_d)$ and $\bar{Y} = (y_1, y_2, \ldots, y_d)$ be vectors in $\mathbb{R}^n$.

$$\mathrm{CosSim}(\bar{X}, \bar{Y}) = \frac{\bar{X}^T \bar{Y}}{\|\bar{X}\|_2 \, \|\bar{Y}\|_2} = \cos(\theta)$$

Where

$$\|\bar{X}\|_2 = \sqrt{\sum_{i=1}^{d} x_i^2}$$

and θ is the angle between the vectors $\bar{X}$ and $\bar{Y}$.

Measuring similarity/distance for numerical data

Cosine similarity

$$\text{CosSim}(\bar{X}, \bar{Y}) = \frac{\bar{X}^T \bar{Y}}{\|\bar{X}\| \, \|\bar{Y}\|} = \cos(\theta)$$

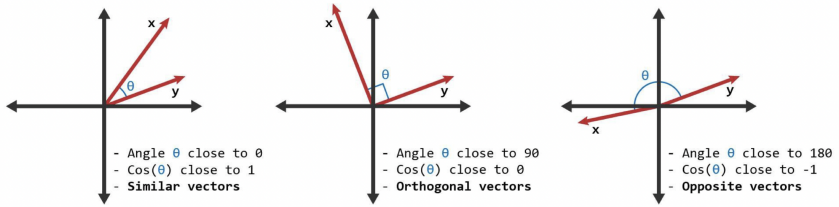


Figure: Demonstration of cosine similarity

Measuring similarity/distance for numerical data

Cosine similarity example (text mining domain):

1. We have a fixed set of **terms (keywords)** of interest
2. Each **term** is assigned a different **dimension**
3. A **document is characterised by a vector** where the value in each dimension corresponds to the frequency of the term appearance in the document
4. **Cosine similarity** then gives a useful measure of how similar two documents are likely to be with respect to their subject matter

Cosine distance

$$\mathrm{CosDist}(\bar{X}, \bar{Y}) = 1 - \mathrm{CosSim}(\bar{X}, \bar{Y})$$

Measures of similarity/distance for set data

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

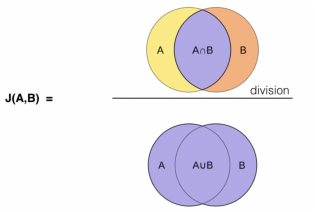


Figure: Diagram of Jaccard Similarity

Jaccard distance: $d_J(A,B) = 1 - J(A,B)$

Measures of similarity/distance for set data

Overlap coefficient:

$$\text{overlap}(A, B) = \frac{|A \cap B|}{\min(|A|, |B|)}$$

Measures of similarity/distance for binary data

Hamming distance is defined for binary vectors of the same length. It is equal to the number of coordinates where the two vectors differ.

$$\text{Hamming}(\mathbf{X}, \mathbf{Y}) = |\{\, i : x_i \neq y_i \,\}|$$

Example:
$\mathbf{X} = (1, 0, 1, 0, 1, 1)^T$
$\mathbf{Y} = (0, 0, 1, 1, 1, 1)^T$
$\text{Hamming}(\mathbf{X}, \mathbf{Y}) = 2$

Measures of similarity/distance for categorical data

Key differences with respect to numerical data:

- no ordering on the values;
- no natural "difference" operation.

One approach to compute similarity between two objects $\mathbf{X} = (x_1, \ldots, x_d)^T$ and $\mathbf{Y} = (y_1, \ldots, y_d)$ with categorical features is to define:

$$\mathrm{Sim}(\mathbf{X}, \mathbf{Y}) = \sum_{i=1}^{d} S(x_i, y_i)$$

where $S(x_i, y_i)$ is the similarity between the feature values x_i and y_i.

Measures of similarity/distance for categorical data

$$\mathrm{Sim}(\mathbf{X}, \mathbf{Y}) = \sum_{i=1}^{d} S(x_i, y_i)$$

Specific $S(x_i, y_i)$ (example 1):

$$S(x_i, y_i) = \begin{cases} 1, & \text{if } x_i = y_i \\ 0, & \text{otherwise} \end{cases}$$

Major drawback:

- does not account for the relative frequencies among different feature values;
- e.g., if the value 'Normal' of a fixed feature appears in 99% of the objects and the rest have values 'Cancer' or 'Diabetes', the fact that two objects have this feature equal to 'Normal' tells us less than if they both would have the feature equal to 'Cancer'

Measures of similarity/distance for categorical data

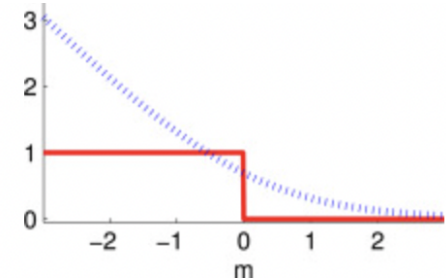
$$\mathrm{Sim}(\mathbf{X}, \mathbf{Y}) = \sum_{i=1}^{d} S(x_i, y_i)$$

Specific $S(x_i, y_i)$ (example 2):
Let $p_k(x)$ be the fraction of objects in which the k-th feature takes on the value x in the data set.

$$S(x_i, y_i) = \begin{cases} \dfrac{1}{p_i(x_i)^2}, & \text{if } x_i = y_i \\ 0, & \text{otherwise} \end{cases}$$

How to Choose K?

Procheta Sen



Effect of K

The Parameter K.

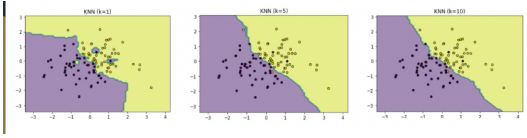


Figure: Effect of Choosing Different Values of K

The parameter k

- k is a *hyperparameter* of the algorithm.
- What value to use for k?
  - Depends on the dataset size. Large and diverse datasets need a higher k, whereas a high k for small datasets might cross out of the class boundaries.
- How to choose k?
  - **A bad option:** try different values of k when evaluating on **test data**.
  - Use **validation dataset** to find a good value of k.

Train / Validation / Test approach

Validation data

- Set aside (hold-out) a fraction of train data for validation purposes.
- Using validation data to set hyperparameters reduces overfitting.
- Of course, you **CANNOT** use test data for any tuning except for testing.

Figure: Train/Test Split

Example of Cross-validation

Figure: K fold Validation

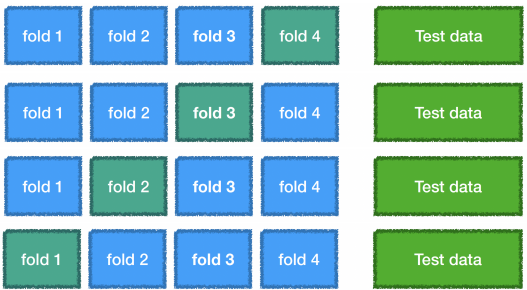
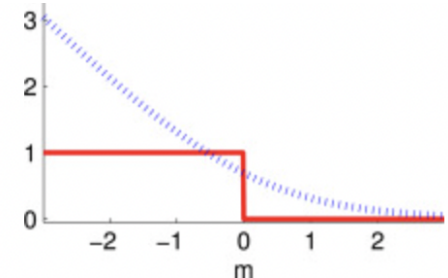


Figure: Demonstration of K fold Validation

Remarks on KNN

Procheta Sen



Complexity of k-NN

- **Training:** store entire training set
- **Classification:** for an input object $\bar{X}'$,
  - find in the training set the k closest (nearest) objects to $\bar{X}'$
  - find the majority label among those nearest neighbours
  - the majority label is predicted to the test instances $\bar{X}'$

Training dataset

n objects and d features
Time: store data (constant per object)
Space: $n \times d$

Test time

For each test example we need to find k nearest neighbours. The test example must be compared with every training example.

Complexity of k-NN (Continued)

- **Training** is computationally cheap
- **Classification** is computationally expensive

In practice the classification time is much more important than the training time

Possible solution: use Approximate Nearest Neighbor (ANN) algorithms that can accelerate the nearest neighbor lookup in a dataset at the expense of query accuracy (e.g., FLANN — Fast Library for Approximate Nearest Neighbors).

Inductive Bias and Feature Importance

k-NN classifier assumes

- that nearby points should have the same label
- all features are equally important!
  (if there are only a few relevant and many irrelevant features in a dataset, the *k*-NN classifier is likely to perform poorly)

Summary on k-NN classifier

- Preprocess your data: normalise the features in your dataset to have zero mean and unit variance.
- If your data is very high-dimensional, consider using dimensionality reduction.
- Split your training data into training (50–90%) and validation (10–50%).
- If the validation dataset is too small, split your training data into folds and perform cross-validation.
- Train and evaluate the k-NN classifier on the validation data for many choices of k and for different types of distances (start with L^1 and L^2).